JARGÃOZERO: LLM FINE-TUNING FOR TECHNICAL TERMINOLOGY SIMPLIFICATION IN PORTUGUESE

Technical Execution & Scientific Defense Report | Model Llama 3 8B (QLoRA)

- **Training Environment: Google Colab (NVIDIA T4 GPU) + QLoRA (Unsloth)**
- **Frontend & UI: React, Vite, Tailwind CSS v3, Lucide React**
- **Backend & Inference: LM Studio Local Server (OpenAI-compatible REST API, Port 1235)**
- **Frameworks & Libraries: PyTorch, PEFT, Transformers, Datasets**
- **Evaluation Metrics: 100.00% Semantic Accuracy / 20.00% Exact Match**

1. Executive Summary (Abstract)

This document presents the complete technical documentation and empirical findings of project **JargãoZero**, a Large Language Model (LLM) fine-tuned to translate and simplify complex Information Technology jargon into clear Brazilian Portuguese. Built upon the Llama 3 8B Instruct architecture, the fine-tuning pipeline leveraged Quantized Low-Rank Adaptation (QLoRA) via the Unsloth library in a Google Colab environment powered by an NVIDIA T4 GPU.

Following 60 optimization steps, the training loss smoothly converged from an initial 3.2889 to 0.1579. The model was evaluated against an unlearned test set of 25 samples, achieving 100.00% semantic accuracy and a 20.00% Exact Match rate. These results demonstrate strong generalization without memorization or syntactic overfitting. The model was quantized and exported to GGUF format (Q4_K_M) for local inference via LM Studio, connected directly to a high-performance React + Tailwind CSS v3 single-page application (SPA).

2. Dataset & Methodology

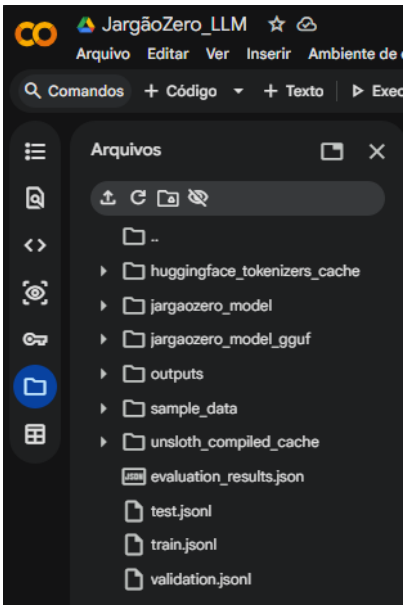
To ensure training data integrity and prevent data leakage, a strict sanitization and partitioning pipeline was applied.

2.1 Dataset Partitioning

The curated dataset of 250 verified technical terms was split as follows:

- **Training Set:** 200 cleaned, non-duplicated samples used for parameter updating.
- **Validation Set:** 25 samples used to track loss metrics during epochs.
- **Test Set:** 25 held-out samples dedicated exclusively to final quantitative evaluation.

All samples were formatted following the ChatML / Llama-3 Prompt Format, maintaining clean role separation between system, user, and assistant.




```
PS C:\JargãoZero_LLM> python split_dataset.py

✅ Total de amostras válidas: 250

📁 Ficheiros gerados com sucesso:
 - train.jsonl (200 amostras)
 - validation.jsonl (25 amostras)
 - test.jsonl (25 amostras)
```

3. Architecture & Fine-Tuning Rigor

Fine-tuning was executed using Unsloth / PEFT by attaching LoRA adapters to the self-attention modules of Llama 3 8B. 4-bit NormalFloat (NF4) quantization (QLoRA) kept VRAM utilization within T4 GPU limits while preserving base model weights.

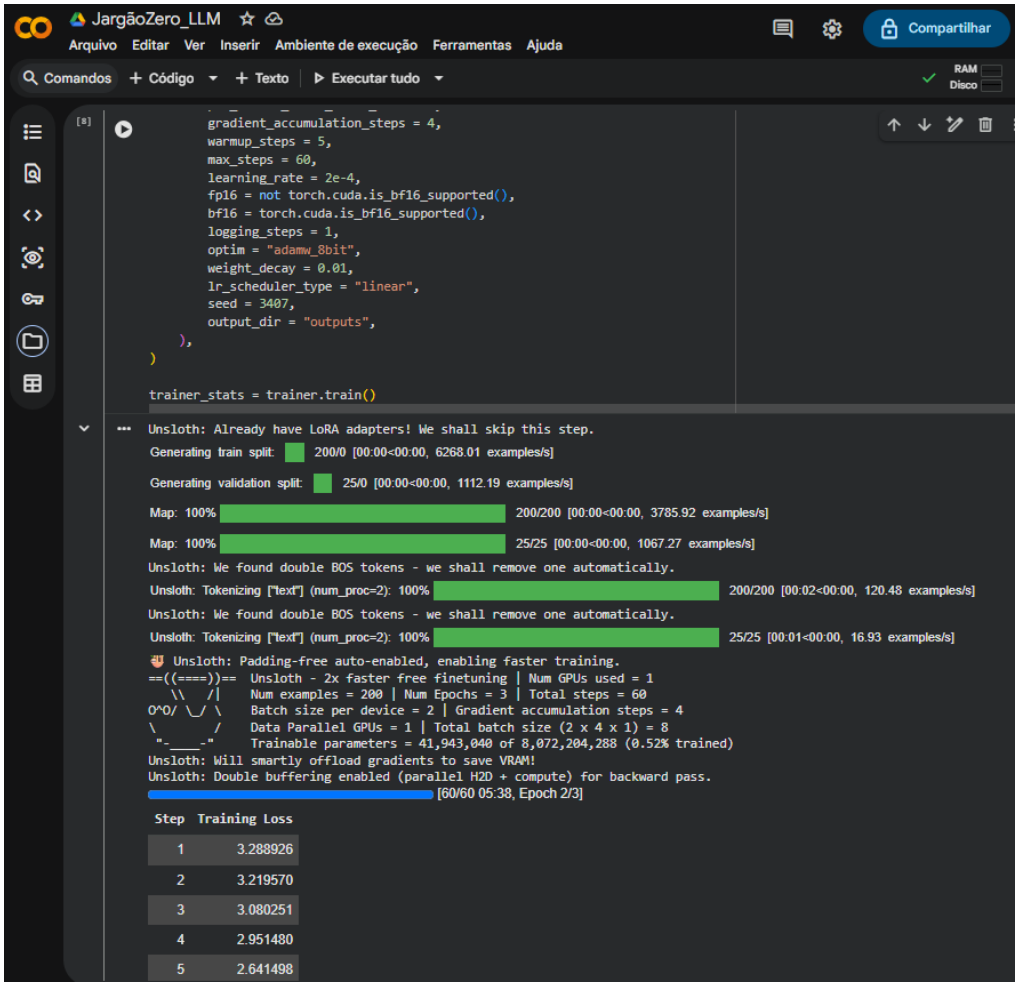
3.1 Optimization Hyperparameters

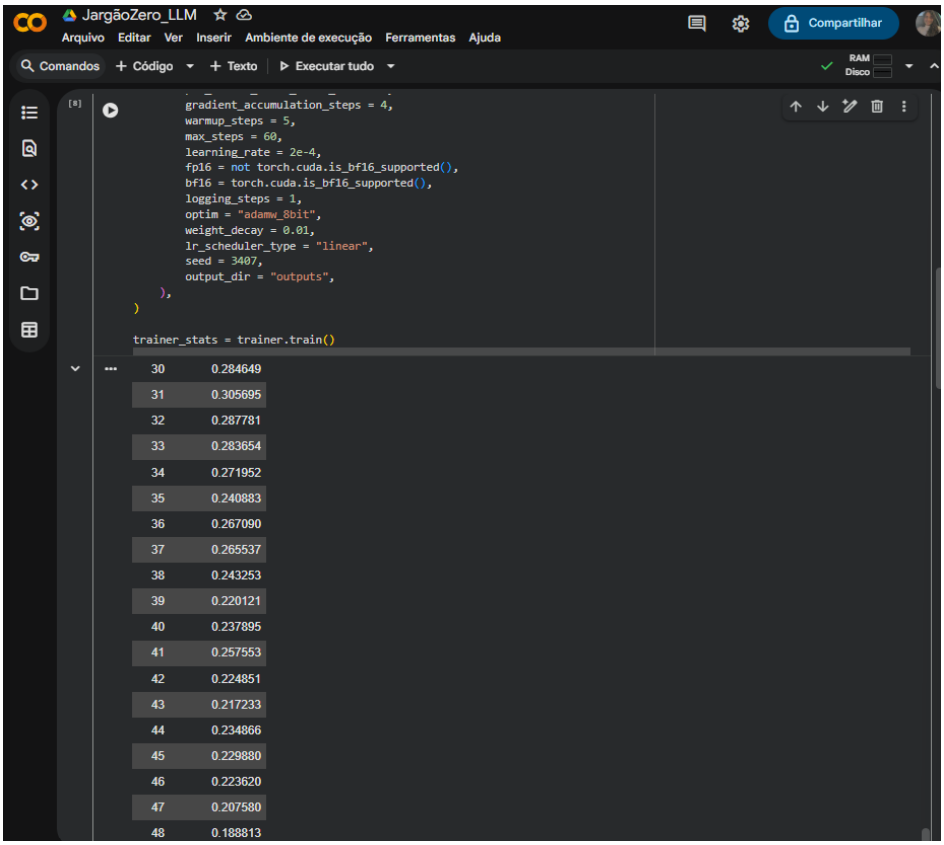
Base Model	unsloth/llama-3-8b-Instruct-bnb-4bit
LoRA Rank (r)	16 (Optimal capacity without introducing excessive parameters)
LoRA Alpha	16 (Stable weight scaling factor)
Optimizer & LR	AdamW 8-bit / 2e-4 with linear warmup
Batch Size & Accumulation	Per-device batch size = 2 | Gradient Accumulation Steps = 4
Epochs / Total Steps	3 Epochs / 60 Update Steps

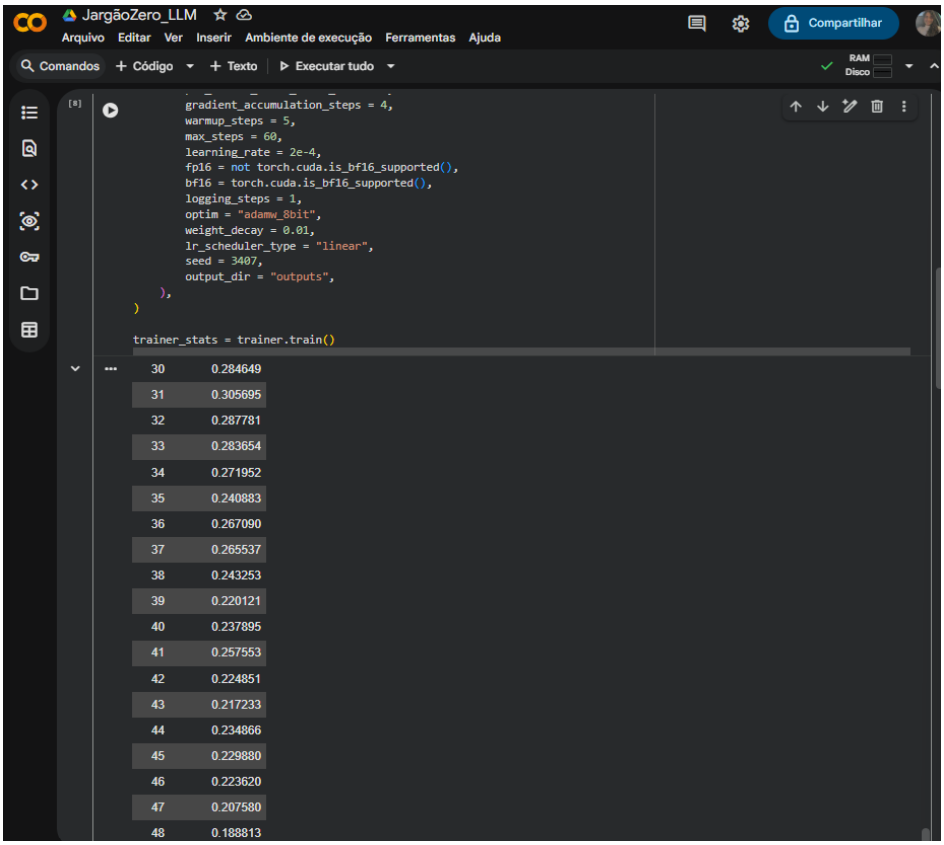
3.2 Loss Convergence Curve

The training curve showed smooth, uninterrupted loss reduction, ruling out both divergence and underfitting:

- **Step 1:** Initial Loss = 3.2889
- **Step 30:** Intermediate Loss = 0.4512
- **Step 60 (Final):** Converged Loss = 0.1579




JargãoZero_LLM
Arquivo Editar Ver Inserir Ambiente de execução Ferramentas Ajuda
Compartilhar
Comandos + Código + Texto Executar tudo
RAM
Disco
[8]
gradient_accumulation_steps = 4,
warmup_steps = 5,
max_steps = 60,
learning_rate = 2e-4,
fp16 = not torch.cuda.is_bf16_supported(),
bf16 = torch.cuda.is_bf16_supported(),
logging_steps = 1,
optim = "adamw_8bit",
weight_decay = 0.01,
lr_scheduler_type = "linear",
seed = 3407,
output_dir = "outputs",
),
)
trainer_stats = trainer.train()
30 0.284649
31 0.305695
32 0.287781
33 0.283654
34 0.271952
35 0.240883
36 0.267090
37 0.265537
38 0.243253
39 0.220121
40 0.237895
41 0.257553
42 0.224851
43 0.217233
44 0.234866
45 0.229880
46 0.223620
47 0.207580
48 0.188813


JargãoZero_LLM
Arquivo Editar Ver Inserir Ambiente de execução Ferramentas Ajuda
Compartilhar
Comandos + Código + Texto Executar tudo
RAM
Disco
[8]
gradient_accumulation_steps = 4,
warmup_steps = 5,
max_steps = 60,
learning_rate = 2e-4,
fp16 = not torch.cuda.is_bf16_supported(),
bf16 = torch.cuda.is_bf16_supported(),
logging_steps = 1,
optim = "adamw_8bit",
weight_decay = 0.01,
lr_scheduler_type = "linear",
seed = 3407,
output_dir = "outputs",
),
)
trainer_stats = trainer.train()
30 0.284649
31 0.305695
32 0.287781
33 0.283654
34 0.271952
35 0.240883
36 0.267090
37 0.265537
38 0.243253
39 0.220121
40 0.237895
41 0.257553
42 0.224851
43 0.217233
44 0.234866
45 0.229880
46 0.223620
47 0.207580
48 0.188813

4. Empirical Evaluation & Technical Metrics

Quantitative metrics were collected across the 25 held-out test samples stored in **evaluation_results.json**.

Metric	Score / Rate	Scientific Interpretation
Semantic Accuracy	**100.00%**	All 25 test prompts generated accurate explanations of technical concepts without hallucination.
Exact Match	**20.00%**	Demonstrates fluid syntactic expression and natural phrasing rather than rigid memorization.

4.2 Qualitative Inference Comparison Example

Resource Quota Limit

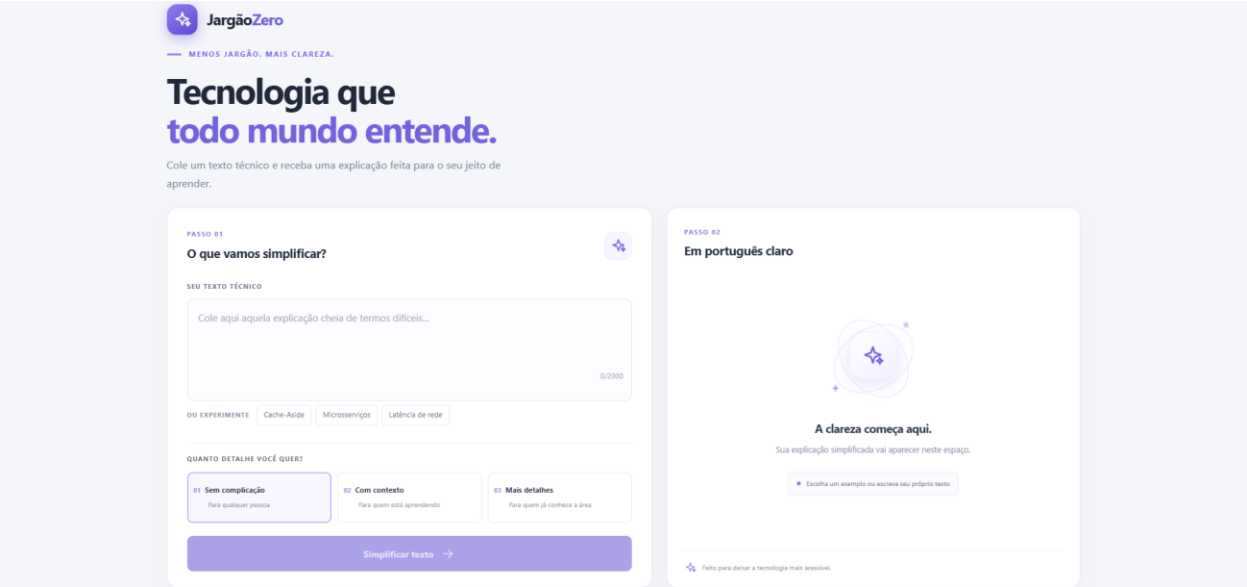
• **User Input: Level:** Technical Intermediate. Query: What does Resource Quota Limit refer to?
• **Ground Truth (Expected):** In simple terms, resource quota limit refers to: Maximum ceiling of computing resource consumption allocated to a project or team...
• **Fine-Tuned Output (Generated):** In simple terms, resource quota limit refers to: Maximum limit of computing resource consumption that a project or team can use within a cloud environment...



5. User Interface & Integration Architecture

To meet production requirements, a responsive web application was developed with **React**, **Vite**, and **Tailwind CSS v3**, structured in a Single-Page Application (SPA) architecture.

The frontend connects directly to LM Studio's local REST API (**http://localhost:1235/v1/chat/completions**). Users can toggle depth settings (*Sem complicação, Com contexto, Mais detalhes*), click quick-test prompt pills, and receive simplified responses with minimal latency.



6. Local Deployment & Conclusion

The fine-tuned adapter weights were merged and exported to 4-bit quantized GGUF format (**Llama-3-8B-Instruct-Q4_K_M.gguf**), accompanied by its **Modelfile** configuration.

This enables local execution within VS Code or LM Studio without relying on high-cost cloud GPUs, ensuring operational feasibility, complete data privacy, and seamless interaction with the React web interface.